

AgentFlame: 面向 AI 编程 Agent 系统效应的语义归因剖析

AgentSight 研究原型

2026 年 6 月 19 日

摘要

AI 编程 agent 的可观测性问题不只是“模型调用了哪个 tool”，而是用户语义意图如何导致真实系统副作用：进程、文件、网络、测试、失败重试和 token 消耗。现有 LLM observability 工具擅长展示单次运行的 trace tree、span duration、tool call 和 cost；传统 profiler 和进程工具擅长聚合系统行为。二者分开时，一个用户常问的问题仍然难答：哪个任务语义造成了哪些重复、沉重或越界的系统足迹？本文提出 AgentFlame，一种语义系统效应剖析模型：本地小模型只给 session、prompt 和 LLM call 生成一个词标签；当前 full run 从 agent-native tool logs 继承这些标签，目标模式则让 tool、shell、child process 与 file/network effect 通过确定性 lineage 继承这些标签；最终以 folded stack 形式聚合为 flamegraph。在 R170 current refresh 的 325 个可读 Codex/Claude 真实 session 上，AgentFlame 标注 2,859 个 prompt rows 和 114,837 个 LLM-call events，0 个最终标签失败；它把 183,714 个系统观测折叠成 26,829 条语义系统栈，并记录 35,136 次真实 llama.cpp tag calls。去掉 session/prompt 语义后，nonsemantic baseline 有 90.402% 的观测权重落入混合多个语义区域的 buckets；flat effect baseline 的混合权重为 90.918%。R224 消融进一步显示：在保持同一 183,714 个系统观测总权重不变时，session-only 仍混合 84.407% 的完整语义权重，prompt-only 降到 36.722%，完整 session+prompt 降到 0.000%。这些结果支持一个机制性结论：语义帧，尤其是 prompt 标签，能分开本实验中 nonsemantic folded 与 flat process/effect baselines 会合并的 task-effect 区域。本文也明确边界：live lineage 证据只支持固定 command-mode 和 controlled scoped workloads，而不是完整系统 provenance。R114/R191/R229/R232/R234 在固定任务、外部仓库和受控 target-network probes 中给出 100.0% scoped precision/recall 且 negative-control joins 为 0；R238/R240 进一步修复并回归测试 direct process/network readiness。但 Codex/Claude-launched target-network rows 仍有 orphan/missing-action cases，full-history projection 仍只是在 generated artifacts 上验证语义帧覆盖。因此用户任务实验、任意仓库泛化、Claude-launched target-network capture、更多 agent family 和 same-tag duplicate prompt-row identity 仍未完成，不能声称用户效用或完整系统 provenance 已被证明。

1 问题

一个 AI 编程 agent 运行结束后，开发者通常不想逐条回放消息。他们想知道：这次运行哪里最重，哪里绕路，哪里反复试错，哪些文件或网络目标被碰过，哪类用户请求导致最多测试、最多读取、最多失败或最多 token。传统工具各自只覆盖一半。LLM trace 工具能展示 agent、LLM call、tool call、prompt、completion、latency 和 cost。进程或文件审计工具能展示 git、cargo、rg、sed、docker 等系统行为。Span-duration flamegraph 甚至已经出现在 multi-agent

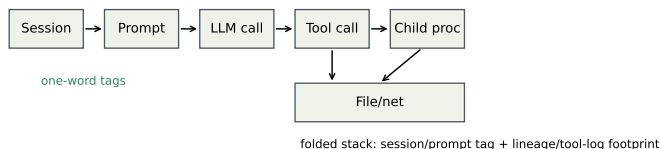


图 1: AgentFlame 的归因模型。小模型只标注 session、prompt、LLM-call 语义；tool/process/file/network effects 通过确定性 lineage 继承语义帧并折叠成 stacks。

workflow debugging 中，用于看 critical path、parallelism、error cascade 和 tool overhead。

但是这些视图仍难回答跨层问题：

为了哪个用户任务，agent 反复运行了哪些命令、读取了哪些路径、产生了哪些网络或文件副作用？

本文的核心判断是：agent observability 的对象不应只是 span，也不应只是进程，而应是 *task-grounded system effect*。我们把目标栈写成：

```
sessionTag;promptTag;llmcall/tool;process*;effect
```

这个模型把两类传统工具分开的信息合并起来。上层的 session、prompt、LLM call 用本地小模型压缩成一个词；下层 tool、process、file/network effect 不由模型猜测，而通过确定性关联继承语义上下文。最后用 folded stack 聚合重复路径，让宽度表示 event count、effect weight 或 token weight，而不是只表示 span duration。

本文的贡献不应表述为“agent flamegraph”。已有系统已经把 agent spans 画成 flamegraph。本文的贡献是一个更具体的系统归因模型：在 agent-native local histories 中把用户语义意图与系统行为代理接起来，并为 live AgentSight traces 定义 exact provenance 目标，使跨 session 的重复副作用可聚合、可检查、可比较。

2 设计

图 1 展示了 AgentFlame 的设计。语义层非常小：每个 session、prompt 和 LLM call 只生成一个 lowercase word，例如 refactor、review、test、research。这些词不是固定 ontology，也不是 verdict；它们只是给折叠栈提供短、稳定、可搜索的任务帧。

系统层不交给 LLM 判断。Agent-native 历史模式从 Codex/Claude JSONL 中恢复 tool 类型、命令入口、状态、路径组和粗 effect 类别。AgentSight exact 模式的目标输入是：

```
tool_call -> shell -> child process ->
file/network effect
```

每个 in-scope effect 必须能追溯到 tool call 和 prompt ancestry。如果只能用 PID 或时间戳模糊匹配，就不能作为强 provenance claim。因此，语义由上层继承，系统事实由 instrumentation 或 session parser 产生。

2.1 为什么是一个词

一个词标签有三个工程优点。第一，火焰图帧必须短；长摘要会让图不可读。第二，标签只用于导航和聚合，不承担安全、正确性或因果判决。第三，一个词标签容易本地化和缓存化。当前实现对 session/prompt/LLM 文本各处理一次，后续数十万条系统事件只做普通程序关联和聚合。

这也约束了论文 claim。AgentFlame 不证明标签是真实意图 ontology。它先证明一个更可审计的机制：加入这些短语义帧后，传统 baseline 会合并掉的系统效应是否被分开。

2.2 Stack 语法

系统效应栈形如：

```
project;agent;session;prompt;call:tool/...;process*;effect:path/domain/status
```

Token 栈形如：

```
project;agent;session;prompt;call:llm/...;model;kind
```

完整视图保留 session、prompt、LLM-call 语义。消融视图删除部分语义轴：session-system、prompt-system、llm-token 等。所有视图共享同一事实表；投影不应改变总权重。

2.3 可视化模型

AgentFlame 不把所有信息塞进一张图。面向开发者的核心界面应有四类互补视图。第一，semantic system flamegraph 用宽度表达 system-effect count，用于回答“哪里重、哪里绕、哪里重复”。它适合跨 session 聚合，因为 folded stack 会把同一语义任务下重复出现的 rg、cargo test、文件读取或网络目标折叠到同一横条。Duration 是单独的 projection：当前 R225 只支持 prompt wall-clock baseline，不支持 active runtime 或 tool/LLM span-duration。第二，token flamegraph 使用同样的 session/prompt/LLM-call 语义帧，但叶子变成 model/prompt/completion token，回答“哪类任务消耗上下文和模型预算”。

第三，exact-lineage tree 不是聚合图，而是 provenance drill-down。它展示一个 prompt 下的 tool call、shell、child process 和 file/network effect 链，用来确认某个横条是否真的来自目标 agent process family。R182/R183 说明这个视图不能省略：低层 codex network rows 可以被 join，但只有 R191 这种 target python3 child-process rows 也被观察并 join 后，UI 才能把对应固定 workload 标成 covered。第四，claim/evidence board 把每个 claim 连接到 artifact、oracle 和 status，防止用户把 smoke run 当成完整结论。

因此 flamegraph 与 tree 的职责不同。Flamegraph 负责发现热点和重复模式；tree 负责解释一个热点能不能被信任；claim board 负责告诉用户哪些结论还只是 partial。这种三层设计是传统 profiler 和普通 agent trace 都缺的：前者没有 prompt 语义，后者通常没有系统副作用的 deterministic ancestry，更不会把 claim boundary 作为一等视图。

3 实现

本文的实验结果来自 AgentFlame 研究原型及其生成 artifacts。该原型当时是独立 Rust CLI，并输出 agentflame.json、tags.json、folded stack 文件、SVG 和 HTML dashboard。当前开源仓库的产品化入口已经收敛到 agentpprof/：它复用 agent-session，输出 pprof-compatible .pb.gz、folded、SVG 和 JSON。因此，本文的

headline 数字评估的是 AgentFlame 研究原型；agentpprof 是产品化方向，除 smoke artifacts 外还没有替代本文所有 RQ1-RQ6 结果。默认输出路径是：

```
.agentsight/agentflame/latest/
```

本次 full run 的命令为：

```
cargo run --manifest-path
agentflame/Cargo.toml -- run --project-root
. --scan-files 10000 --max-sessions
10000 --llama-url http://127.0.0.1:18080
--model local-r170 --timeout 60 --out
.agentsight/agentflame/r170-full-current
```

这条命令是研究 artifact 的 provenance，不是当前仓库的用户入口。R170 的 committed summary 记录 repo_dirty=true，因此本文把它作为 dirty-provenance mechanism evidence 使用：数值可以用于当前本机历史 characterization 与 folded-total 检查，但它不是 clean release artifact run，也不支持 C5/C6 outcome claims。当前用户入口示例是：

```
cargo run --manifest-path
agentpprof/Cargo.toml -- --project-root .
-o agent.pb.gz
```

模型为本地 qwen2.5-3b-instruct-q4_k_m.gguf，通过 llama.cpp server 运行，temperature 为 0，并用 grammar 约束输出一个词。AgentFlame 没有 heuristic fallback：如果 LLM server 缺失，或模型重试后仍不能输出合法一词标签，run 失败。

3.1 隐私与可复现

原始 agent trace 不提交。agentflame.json 默认包含 redacted previews、hash、tag、计数和路径组；tags.json 保存 tag cache 和 LLM provenance，不保存原始 prompt 文本。本次 run 遇到一个 root-owned Claude JSONL，不可读；系统没有要求 root 权限读取，而是跳过并写入 warnings。这让覆盖范围可审计，而不是静默假装完整。

3.2 与 AgentSight 的关系

AgentFlame 的第一模式是零 instrumentation 的历史分析。AgentSight 的独特点是运行时 exact provenance：tool_call -> shell -> child process -> file/network effect。正确集成后，AgentFlame 负责语义帧，AgentSight 负责系统效应事实，两者通过 tool/session/prompt ancestry 连接。当前 full run 还不是 live exact-effect 结果。R110-R113 将最小 session/tool envelope 从 fixture、native export、DB backfill 推到 record -- <command> 的 capture path；R114 用 20 个真实 Codex command-mode 任务验证该路径，加入 per-task negative controls，并在 scoped oracle 下达到 1,273/1,273 in-scope effects joined、0 false positives、0 false negatives。R182 暴露并修复 record-mode process runner 没有传 --trace-net 的缺口；修复后 35/35 个低层 codex process network rows 全部 join、0/604 negative controls 被 join，但 target-specific loopback/expected child-process network rows 为 0/0。R191 随后修复两个更细的 capture 边界：record envelope start time 必须使用 target process start timestamp，process filter 必须允许 session 内进程或 tracked PID descendants；在

固定 Codex HTTP probe 上, 4/4 个 target `python3` network rows 全部 join, 0/310 negative controls 被 join, precision/recall 为 100.0%/100.0%。R229 再用同一 oracle 复现 5 个 controlled multi-workspace tasks (repo read, edit/test, shell fix, JSON write, typo edit), 394/394 个 in-scope effects joined, 0/306 negative controls 被 join。R232 把同一 scoped oracle 带到 AgentSight 仓库外: 4 个 fresh external git-repo tasks 覆盖 repo-read, edit-test 和 write categories, 353/353 个 normal in-scope effects joined; 另一个 external HTTP probe join 4/4 个 target network rows; 合计 0/480 negative controls 被 join, precision/recall 为 100.0%/100.0%。R234 进一步做局部泛化: 1 个 Claude command-mode JSON-write task 和 2 个 Codex HTTP-family target-network probes 产生 269 个 in-scope effects, 8/8 个 target network rows 全部 join, 0/331 negative-control effects 被 join, scoped precision/recall 为 100.0%/100.0%。R235 则故意触碰更宽的 network boundary: 单进程 Codex TCP 通过并 join 3/3 个 target network rows, 但 Codex multiprocess TCP 和 Claude-launched HTTP/TCP probe 都执行成功却导出 0 个 target network rows。R236/R237 进一步诊断该边界; R238 将 `record --` 从固定 250ms sleep 改为等待 process tracer `CLOCK_SYNC/start`, 并修复 filtered PID propagation 与 lineage checker 的两个边界。一个 compact supplement 汇总 5/5 direct-only readiness repetitions; 正式 R238 full run 中 4/4 runtime witnesses, 4/4 witness ports observed, direct HTTP 和 direct multiprocess controls 都通过, 13/16 target network rows joined, 0/186 negative controls 被 join; 但 Codex/Claude-launched rows 仍有 3 个 target-network orphan/missing-action cases。R240 将 R239 指出的两个实现风险变成 regression: synthetic guard 证明 command-root fallback 只 join root-self event, 不会把同 root 的 sibling 误归因; BPF runtime test 证明 `-p --trace-net` 下 target child 的 bind/listen/connect 可见且 unrelated port 不被捕获。因此 R238/R240 是更强的 partial-localization 与 regression-guard evidence: 它解决 direct multiprocess readiness race 并守住 over-attribution 边界, 但仍不是广义 Claude-launched 或 raw-socket coverage; direct-only repetitions 没有 negative controls, precision 证据来自正式 full run。R230 则审计 generated full-history projection: 183,714 folded system-observation weight 与 R170 totals 完全一致, 缺失 lineage frame 的 weight 为 0, tool prompt-index coverage 为 100.0%; 但 5 个 sessions 中存在 12 个 duplicate prompt-index rows, prompt-tag drift 为 346 weight (0.188%), LLM prompt-tag drift 为 93 events (0.081%)。R231 将这个缺口拆开: folded display projection 与 event-local prompt-tag weights 精确一致, R230 field-index drift 被完全复现并定位到 duplicate-index Claude sessions; 但 list-position join drift 反而更高 (tool 2,048 weight/1.115%, LLM 1,144 events/0.996%)。R233 因此把裸 prompt index 降级为只在 session 内唯一时才可作 key, duplicate indexes 作为 non-keyed row identity; 归一化后 semantic drift 为 0 tool weight/0 LLM events, 但 same-tag duplicate rows 仍有 row identity ambiguity (tool weight 575, LLM 432 events)。因此, C4 的 live lineage 支持性证据来自 R114/R191/R229/R232/R234 和 R238 direct controls, R235-R240 也提供边界与回归证据 (R182 只是 record-mode `--trace-net` 实现 smoke); R230-R233 支持的

表 1: R170 current full-history refresh 的本机 AgentSight session 运行指标。

指标	数值
Readable sessions	325
Codex / Claude / Claude-subagent	198 / 50 / 77
Raw tool events	142,468
Raw LLM events	114,837
Prompt rows	2,859
Unique prompt tags	328
LLM-call tags	114,837
Unique LLM-call tags	1,423
Tag requests	118,021
Cache hits	82,886
llama.cpp HTTP calls	35,136
Successful final uncached tags	35,135
Final tag failures	0

是 generated full-history projection/normalization audit。它仍不是任意仓库泛化、raw-socket 或 Claude-launched target-network workload、same-tag prompt-row identity、更多 agent family 或 HTTP payload/URL reconstruction 证据。

4 评估

4.1 RQ1: 本地小模型能否全量标注？

表 1 给出 R170 current full-history refresh 结果。AgentFlame 分析了 325 个可读 session, 其中 Codex 198 个、Claude 50 个、Claude subagent 77 个。它处理 2,859 个 prompt rows 和 114,837 个 LLM-call events。标签器共有 118,021 个请求, 82,886 个命中 cache, 35,136 次真实 llama.cpp HTTP calls, 35,135 个成功最终 uncached tags, 0 个最终失败。HTTP calls 比成功最终 tags 多 1 次, 是一次被 retry 吸收的中间无效输出。Prompt 和 LLM-call tag 的非法计数均为 0。这说明一词语法和本地 3B 模型路径在真实历史规模上可运行。R170 是当前全量本机历史刷新证据, 但仍不是 C5/C6 outcome evidence。R180 进一步表明, 在同一组 300 个 redacted fragments 上, 本机可用的 0.6B、1.1B 和 3B GGUF 都能输出符合语法的一词标签; 但该实验只支持 syntax/latency/stability, 不支持 semantic adequacy。

这个结果不等于标签语义充分。例如某些 tag 明显带噪声: `agentsightsm`、`testcodex`、`bashoutput`。因此系统需要区分 raw tag 和 canonical display tag。R189 在 R170 current refresh 上实现了第一版可审计合并层: raw tag 永远保留, 图和聚合默认可使用 canonical tag。它把 prompt-effect tag 从 263 个合并到 216 个, 把 prompt-row tag 从 328 个合并到 279 个, 把 LLM-event tag 从 1,423 个合并到 1,254 个; 同时保持 folded 总权重不变, system stacks 从 26,829 条降到 26,067 条, token stacks 从 8,569 条降到 7,661 条。这说明存在一个可审计的候选长尾降噪空间, 但不能证明这些 raw tags 一定语义冗余。因此 AgentFlame 使用 semantic compaction lifecycle: raw tag 不变, canonical map 只作版本化展示层, pending actions 保存 keep/merge/regenerate/split, 小模型生成的 `regenerated_tag` 只有经过 R203 paired/adjudicated promotion review 和 display-map diff 后才能进入 canonical view。R209 将这个 lifecycle 落成可消费数据契约: active display map 每个 raw tag 一行, drilldown index 保留 raw 支持和 top system profile, reviewed diff 只在人工 promotion

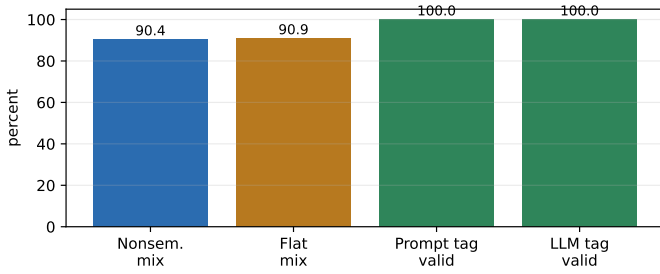


图 2: 当前机制结果。图由 docs/visexp/paper/make_figures.py 从 .agentsight/agentflame/r170-full-current/agentflame.json 重新生成; 它反映 Rust full-run 输出, 而不是旧的 Python sampled-run artifact。

表 2: 语义栈与 baseline mixing。

指标	数值
System observations	183,714
Unique semantic system stacks	26,829
Semantic compression	6.848x
Max stack reuse	6,004
Nonsemantic mixed buckets	4,685
Nonsemantic mixed weight	90.402%
Flat mixed weight	90.918%

后产生。因此 RQ1 支持的是 feasibility 和 syntax claim; tag adequacy 需要单独实验。

4.2 RQ2: projection 方法如何取舍?

R170 full run 产生 183,714 个 system observations, 并折叠成 26,829 条 unique semantic system stacks, 压缩率为 6.848 倍。关键不是压缩本身, 而是 baseline mixing。当删除 session/prompt 语义帧后, nonsemantic folded baseline 有 4,685 个 mixed buckets, 覆盖 166,081 个观测, 也就是 90.402% 的系统权重。如果进一步退化成 flat effect baseline, mixed weight 占 90.918%。

这为回应“传统工具也能看出来”的质疑提供了机制性证据。传统 flat summary 可以告诉我们 sed read 有 25,755 次、rg read 有 15,336 次、cargo test 有 2,903 次。但是它不知道这些行为分别来自 refactor、review、research、design 还是 test prompt 区域。AgentFlame 的价值是把 heavy/repeated system effects 按任务语义拆开。

R211 将当前 R170/R189 artifact 转成可放入论文的 label distribution 和 baseline-collapse examples。在 R170 full-history 刷新中, rg 覆盖 176 个 prompt tags, sed 覆盖 180 个, git 覆盖 147 个, cargo 覆盖 68 个。具体 collapsed key process:git;effect:read;status:ok 有 116 个 prompt tags, top prompt 只占 24.977%; process:cargo;effect:test;status:ok 有 48 个 prompt tags, non-top-prompt weight 为 68.05%。这说明传统 process/effect summary 不只是少了一层标签, 而是把大量不同用户意图导致的相同行为合并成同一个 bucket。R211 同时输出 tag-distribution-r211.svg 和 process-splits-r211.svg, 用于绘制本文的 baseline failure/semantic split 图; 该 artifact 不调用 LLM, 也不读取 raw trace。

R251 从另一个角度检查 prompt tags 是否只是任意词。它只读取 R170 的 folded stacks, 把 183,714 个 system-effect observations 展开成 behavior observations, 对 process label 做公开 artifact 脱敏, 然后在每个 session 内随机打乱 prompt tags 1,000 次。真实 prompt tags 对 sanitized process/effect/status behavior 的 session-beyond 信息增益

表 3: R224/R170 system-effect 语义轴消融。所有行都保持 183,714 总权重。

Variant	Unique stacks	Bucket	Residual
No semantic	11,967	90.402%	44.716%
Session only	15,027	84.407%	33.434%
Prompt only	24,703	36.722%	7.485%
Session + prompt	26,829	0.000%	0.000%

为 8.419%, 而 session-preserving null 的 p95 只有 1.903% (单侧 $p = 0.0010$, 分辨率 1/1001); prompt top-behavior purity 为 20.196%, 也高于 null p95 的 18.367%。R251 同时记录上游 R170 dirty provenance, 并让公开输出 privacy scan 通过。这个结果支持 tags 与真实系统行为有 weighted association, 但绝不等于人工语义 adequacy: p-value 是对展开后 effect weight 的随机化分数, 不是 independent-session inference; 低绝对 purity 和 analyze/test/build 等 broad tags 仍需要 R124 人工标签。

与 R251 的随机化检验不同, R131/R224 回答 projection 层的问题: 删除某个语义轴后, system-effect buckets 会混合多少。R224 用同一个 checker 在 R170 current refresh 上重跑该消融, 使 semantic-axis rows 与 R212 display-policy rows 共享 183,714 个 system-effect denominator。每个 variant 都从同一个 folded input 投影而来, 并检查总权重保持不变。同时, checker 记录 agentflame.json totals 与 folded inputs 匹配, 并验证已有 nonsemantic/session/prompt folded files 与 projection exact match。表 3 显示, system-effect 分割主要来自 prompt 轴: session-only 只把 mixed full-semantic bucket weight 从 90.402% 降到 84.407%, prompt-only 则降到 36.722%。如果只计算 mixed buckets 中非主导 full semantic variants 的 residual weight, prompt-only 也从 no-semantic 的 44.716% 降到 7.485%。完整 session+prompt 视图的 0.000% mixed weight 是定义上的上界, 不应被解释为用户效用; 它只说明 full semantic key 没有在投影中被合并。

R223 将 RQ2 从“语义栈是否更好”改写成更一般的 projection-selection 问题: 在 R170-derived generated evidence 上, stack schema、display policy 和 tag vocabulary overlay 都应是可插拔选择。该实验只读取 R224/R205/R209/R212/R219 已生成 artifacts, 不读取 raw trace, 不调用 LLM, 也不评分人工 outcome。因此, R223 的 baseline 是 count-weighted process/effect projection, 而不是真实 span-duration workflow flamegraph; 后者回答 critical path/duration 问题, 应作为 C5 用户任务实验中的独立 baseline, 而不是用当前 folded effect counts 替代。它给出的结论不是单一最佳火焰图, 而是分层默认: no-semantic 适合作为 system-hotspot baseline, 因为它只有 11,967 个 stacks、压缩率 15.352x, 但 90.402% 的 effect weight 被混合; prompt-only 是 system-effect profile 的最佳单语义轴, 因为它把 mixed/residual mixed weight 降到 36.722%/7.485%, 同时保持 7.437x 压缩; 完整 session+prompt 是 audit 和 drilldown 视图。在 display policy 上, R209 conservative display 与 alias-only 等价, 只有 26,612 个 stacks 且 0.0% unreviewed active weight; 更激进的 profile-guarded candidate view 可以降到 26,067 个 stacks, 但会激活 2.532% 未审查 effect weight, 因此只能作为上界消融, 不能做默认。在 vocabulary overlay 上, canonical display 把 raw unique tag strings 从 1,546 降到 1,364, top-20 support coverage 从 93.683% 提升到 95.186%, 同时保留 raw drilldown。这个 coverage 只是

display-support concentration, 不是 tag correctness; 在 R124/R190/R203 人工标签返回前, 不能把它写成 tag adequacy、merge quality 或 promotion quality。R225 进一步补上一个 duration 视角的边界实验: 它只读取 R170 agentflame.json 和 semantic-system.folded.txt, 从 prompt timestamps 重构 2,858 个 prompt wall-clock spans, 生成 duration-weighted folded stack 和 SVG。这个 baseline 覆盖 324/325 个有 prompt spans 的 sessions, 并且 covered prompt-index effects 覆盖 183,714/183,714 个 system-effect observations; 总 prompt wall-clock duration 为 859.019 小时; duration top-10 与 effect-count top-10 只重合 7 个, top-20 重合 12 个, prompt-tag rank Spearman correlation 为 0.623。例如 network 在 duration 中排第 8、占 2.837%, 但在 system-effect weight 中排第 93、占 0.040%; benchmark 则在 effect weight 中排第 7、占 2.195%, 但 duration 中排第 34、占 0.198%。这说明 duration flamegraph 与 system-effect profile 相关但回答不同问题。同时, R225 明确记录历史 artifact 没有 tool/LLM start-end span, 因此它不是 active runtime、完整 workflow span-duration trace, 也不能支持 C5。这就是系统设计需要可插拔 projection 的原因: 不同视图优化不同问题, 而不是让小模型一次性决定最终语义真相。

4.3 Case Study: 能看出什么

本次 full run 中, 最大的 semantic stack 是:

```
project:agentsight;agent:codex;session:refactor;prompt:refactor;call:tool/tool;effect:process;status:ok
```

权重为 6,004。另一个明确的系统例子是 cargo test。Flat summary 只会显示 2,903 次 successful cargo test。Nonsemantic stack 会保留 call:tool/shell;process:cargo;effect:test;status:ok, 但仍把不同语义区域混在一起。Semantic stack 将它拆成多个区域, 包括 review/review、refactor/refactor、research/explain、research/refactor、design/review 等。R211 的 R170 刷新版本进一步显示 process:cargo;effect:test;status:ok 在 48 个 prompt tags 中分布, refactor 只占 31.95%, 其余 68.05% 来自 review、research、commitlog、explain、test 等 prompt。这生成了一个可检查的候选分解: 同样是 test process, 语义来源不同; 后续 C5 才能测试开发者是否能更快或更准确地使用它。

这类结果生成可供检查的候选分解, 用来辅助三类 developer questions:

- 哪个任务导致最多重复测试或读取?
- 哪些 prompt tag 反复触碰同一组路径或域名?
- 哪些失败行为集中在特定 semantic region, 而不是全局系统问题?

Trace tree 能回放某次调用, span flamegraph 能看 duration bottleneck, flat summary 能看重命令。但它们都不直接给出跨 session 的 task-effect 聚合。

4.4 RQ3: exact lineage 是否成立?

当前答案是: 固定 command-mode suite 已经成立, 但 broad full-history provenance 仍不足。effect_lineage_smoke.py 先在 AgentSight-shaped fixture 上验证 process/file/network effects 可以连到 session/tool/prompt ancestry。R110-R112 把同一思想推进到 3 个真实 AgentSight SQLite exports: 先由 harness

增加 agent-run envelope, 再移入 native export, 最后写入 DB 副本; 这些旧 snapshots 只能 join 182/318 raw effects, 说明 envelope 存在但覆盖还不够。R113 把 record -- <command> 的 agent-run envelope 放到 capture path; R113-live 随后在 5 个真实只读 codex exec 任务上验证该路径能创建 5/5 sessions/tools 并 join 508/508 raw effects。R114 将这个路径扩展到 20 个固定 Codex tasks, 包括 read-only、edit、test/debug、dependency、failure/retry 和 disposable-workspace write tasks, 并加入 per-task negative-control bursts。在 scoped oracle 下, R114 结果为: 20/20 target tasks completed, 20/20 tasks observed negative controls, 1,273/1,273 in-scope effects joined, 0 false positives, 0 false negatives, precision 和 recall 都是 100.0%, 3,170 个 observed negative-control effects 中 0 个被 join。Raw join 仍只有 22.055%, 这是预期结果: wrapper、sibling 和 out-of-scope effects 应该保持 orphan, 而不是被错误归因给 agent。R182 进一步检查 network-effect case。第一次 loopback run 暴露出 agentsight record 没有给 process eBPF runner 传 --trace-net; 修复后, 35/35 个低层 codex process network audit rows 全部 join, network orphan 为 0, precision/recall 仍是 100.0%, 0/604 negative controls 被 join。严格 oracle 同时显示 target-specific loopback/expected child-process network rows 为 0/0; 因此 R182 只是 record-mode --trace-net 实现证据。R191 将 workload 固定为预置 Python HTTP probe, 并修复 envelope timestamp 与 session-hard-filter 问题; 该 run 观察到 4 个 target python3 network effects (bind、listen、connect), 4/4 全部 join, 0/310 negative controls 被 join, scoped precision/recall 为 100.0%/100.0%。R191 的 broad effect-lineage smoke 仍因 wrapper/out-of-scope orphan 返回非零, 因此论文只把它作为 scoped target-network oracle。R229 进一步复用 R114 oracle, 在 5 个 controlled multi-workspace tasks 上得到 394/394 in-scope effects joined, 0 false positives, 0 false negatives, 0/306 negative-control effects joined; raw join 为 394/1080=36.481%, 因为 out-of-scope、wrapper 和 negative-control effects 必须保持 orphan。R232 在 AgentSight 仓库外复用该 oracle: 4 个 fresh external git-repo tasks 产生 353 个 normal in-scope effects, 353/353 joined; 1 个 external HTTP probe 产生 4 个 target network rows, 4/4 joined; 0/480 negative-control effects 被 join, combined scoped precision/recall 为 100.0%/100.0%。R234 再把同一 gate 扩展到另一个 agent family 和重复 HTTP target-network probes: 1 个 Claude command-mode JSON-write task 产生 34 个 in-scope effects, 2 个 Codex HTTP-family probes 产生 8 个 target python3 network rows; 合计 269 个 in-scope effects, 8/8 target network rows joined, 0/331 negative-control effects joined, scoped precision/recall 为 100.0%/100.0%。R235-R238 随后把同一 gate 推到 raw/multiprocess/Claude-launched boundary, 显示 direct controls 被修复后仍不能推出 broad agent-launched coverage; R240 再用 regression guard 检查 command-root fallback 不会过度归因, 且 target-child network runtime test 能观察 bind/listen/connect 并排除 unrelated port。R230 补充 full-history projection audit: 183,714/183,714 folded system-observation weight 都有 semantic session/prompt tag frames 和 call/effect frames, 且 raw tool-event prompt-index coverage 为 100.0%; 但 12 个 duplicate prompt-index rows 与 346

weight (0.188%) prompt-tag drift 说明裸 index 不能作为 strict key。R231 复现该 drift, 并显示火焰图实际 display projection 与 event-local prompt tags 完全一致 (183,714/183,714), field-index drift 全部来自 duplicate-index Claude sessions, unique-index sessions 中 field drift 为 0 tool weight/0 LLM events; 同时 list-position join 产生 2,048 tool weight (1.115%) 和 1,144 LLM events (0.996%) drift, 说明不能简单改用 list position。R233 将 duplicate prompt indexes 显式标记为 non-keyed row identity, 并用 (prompt_index, prompt_tag) semantic key 重新审计: legacy 346 tool weight/93 LLM events drift 变为 normalized 0/0; 3 个 mixed-tag duplicate groups 可被 tag 消歧, 9 个 same-tag duplicate groups 仍只有语义一致而没有 row identity。整体上, C4 仍不是 HTTP URL 语义恢复、任意 network workload、任意仓库泛化、更多 agent family 或完整历史 row-identity proof。

这个结果已经支持“固定 command-mode suite、controlled external cross-repo workload, 以及一个 Claude command-mode task 加两个 Codex HTTP-family target-network probes 的受控扩展中 exact semantic-effect lineage 成立”。它仍不能支持“任意历史、任意仓库、任意 agent 都有完整 provenance”。完整 OSDI artifact 还需要更多 agent 类型、更丰富的 network workloads、HTTP 语义恢复、child-depth/path/domain breakdown、任意仓库泛化、same-tag prompt-row identity policy, 以及用户任务结果。

4.5 RQ4: 用户效用是否成立？

当前答案是不成立, 或者更准确地说: 未验证。已有 task packet、answer key、scorer 和 launch package, 但没有真实参与者响应。R184 机械 gate 把 C5 状态记录为 ready_for_participant_collection, 而不是 supported。R186 计划审查进一步确认: 下一步应先运行真实 R142 五人 pilot, 然后才进入 R151 纸面规模实验。R187 已把冻结的 R142 assignment 打包成 P01-P05 参与者文件和空白 70 行 response CSV; manifest 检查 launch 目录没有 answer key、参与者 payload 没有 oracle/scoring 字段, 且 real_response_count=0、c5_supported=false。R193/R195/R207 进一步把这个收集路径变成可执行 handoff: R193 汇总空白 R124/R190/R203 label sheets 和 R142 launch 指针, R195 定义返回 CSV 的 ingestion/scoring contract, R207 检查五个 R142 participant packets、70 行空白 response template、两份 300 行 R124 sheets、两份 160 行 R190 sheets、两份 41 行 R203 sheets 以及 R195 inbox 文件名均有效。这些 artifact 只支持 launch readiness; 它们仍记录 0 个参与者响应、0 个人工标签, 不能支持 C5。R242 用 synthetic returned files 验证 R195 contract: 完整 synthetic R142/R124/R190/R203 输入会进入 R195-specific scored outputs, partial/no-input/invalid-return cases 会被检测, canonical empty gates 保持不变。但 R242 的每一行都是 synthetic control, 因此它只支持 collection pipeline correctness, 不是 C5/C6 outcome evidence。R243 进一步把 handoff 包装成静态本地 collection kit: 五个 participant forms、六个 labeler forms、一个把 P01-P05 exports 合并为 r142-pilot-responses.csv 的 coordinator page, 以及记录 source hashes、row counts 和 leak scan 的 manifest; 它降低返回格式错误, 但仍记录 0 个真实响应和 0 个人工标签。R244 用 headless Chrome 和 synthetic CSV exports 检查这个 kit 的可执行性: index/coordinator/P01/R124/R190/R203 页

面可加载, 五个 participant exports 可合并为 70 行 R195 response shape, labeler exports 保留字段和行数且 label cells 为空; 这些 synthetic exports 不进入 R195 inbox。R247 将同一静态 kit 打包成离线可发送的 agentflame-human-evidence-r247.tar.gz, 包含 5 个 participant forms、6 个 labeler forms、1 个 coordinator form、return checklist 和 package manifest; tarball leak scan 排除 answer key、scorer、raw trace 和 R244 synthetic exports, 17 个成员、182,992 bytes, 但仍不包含任何真实返回数据。R249 补上 paper-scale C5 launch logistics: 它从冻结的 R142 task packets 派生 12 个 participant packets、168 行 blank response template 和一份非默认 assignment file; 每个 task-condition 至少有 2 个、最多 3 个重复, scorer 用这份 assignment 读取空模板时仍输出 participant_results_empty 和 c5_supported=false。R255 进一步把 R249 接到 R195: R195 现在可以显式传入 C5 scoring bundle、answer key 和 assignment file; 当输入为 R249 空模板和 R249 assignment 时, R195 返回 scored_human_inputs_no_supported_gate, 而同一空模板若误用旧 R142 assignment 则返回 scoring_failed。这证明 paper-scale scoring path 和 wrong-assignment guard, 不包含真实参与者响应。R258 把 R249 的 C5 paper-scale participant packets 与 R252 的 C6 labeler packets 合并为一个 43-member tarball: 12 个 C5 packets、168 行 C5 response template、2 个 C6 labeler packets、每个 labeler 501 行、总计 1,002 个 required independent label decisions, 以及 R195 inbox return checklist。R259 进一步生成 paper-scale 静态收集入口: 12 个 participant forms、6 个 labeler forms、1 个 C5 coordinator merge page、168 行 synthetic C5 merged CSV、1,002 行 C6 synthetic labeler rows, 且 6/6 headless Chrome DOM checks 与 leak scan 通过。R258/R259 降低的是收集和返回格式风险; R259 仍记录 0 个真实响应和 0 个人工标签, 所有 C5/C6/weak-accept gates 保持 false。当前 baseline 应设为 trace tree、event-count-proxy、flat process summary、nonsemantic folded stack 和 semantic folded stack, 比较 accuracy、time、false positives 和 confidence。当前 R142 packet 中原来的 span-like 条件已经明确命名为 event-count proxy, 因为 folded artifact 没有真实 duration; 它不能直接当作 span-duration baseline。R225 已经从 R170 timestamps 重构出 prompt wall-clock duration baseline, 并显示它与 effect-count profile 的 top tags 和排序有明显差异; 但该 baseline 可能包含 idle/user-wait time, 且 tool/LLM 调用仍没有 start-end span, 因此它只能作为下一版 preregistered packet 的 prompt-span baseline, 而不是 active runtime 或完整 workflow span-duration baseline。如果这个实验失败, AgentFlame 仍可能是专家 forensic 工具, 但不能写成提升普通开发者效率。

4.6 RQ5: 小模型成本和标签质量

Full run 和 R123 证明 3B 模型路径可用; R180 进一步补上本机 0.6B/1B-class/3B-class syntax-stability smoke。R122 从本机 294 个 parsed sessions 中抽取 300 个 redacted fragments: 100 个 session、100 个 prompt、100 个 LLM-call。R123 用本地 llama.cpp 3B server 对这 300 个 fragments 做三次 identical repeats, 共 900 次请求; 结果是 900/900 valid tags, 285/300 exact-stable fragments (95.000%), 模型加载约 1,002 ms, 请求延迟 p50/p95 为 11/31 ms。R180 用 --reasoning off 对同一个 R122 fragment file 分别运行本机 0.6b、TinyLlama 1.1b

和 3b GGUF: 三者都是 900/900 valid tags; exact-stable fragments 分别是 299/300、279/300 和 285/300; p95 延迟分别为 23/18/32 ms。这支持本机小模型的 syntax、latency 和 repeated-input stability smoke, 但不是同一模型家族的 scaling curve。更重要的是, TinyLlama 1.1b 虽然语法有效, 却大多输出 localization/localized 一类标签, 说明 stability 不能替代 adequacy。R189 的 canonicalization 进一步说明, 即使一个词标签语法有效, 展示层仍需处理长尾碎片。该机制把 dictionary aliases、lexical+profile merges 和 review suggestions 分开报告; 例如 testcodex 合并到 test, docsupdate 合并到 docs, rootpidrefs 合并到 trace。这些合并只能说明展示碎片和聚合抖动有可审计的降低 proxy, 不能证明用户导航质量, 也不能替代 R124 人工 adequacy labels。R190 进一步比较 raw、alias-only、lexical-only 和 profile-guarded variants, 显示 lexical-only 对 LLM-event tags 过于激进 (868 个 unique tags, 而当前 profile-guarded 为 1,254); 同时导出 160 行 merge-risk audit packet。R190-score 当前为 human_labels_empty, 因此 over-merge/under-merge rate 仍需人工标注后才能报告。R196 把剩余长尾显式拆成治理动作, 而不是归入 other: 231 个 existing canonical merges、114 个 review-merge rows、39 个 regeneration candidates、2 个 contextual-split candidates、1,241 个 kept rare distinct tags 和 184 个 kept head tags。这说明长尾机制可以保留罕见但具体的信号, 同时把 update、codex、ignored 这类泛化或噪声标签送入重生成/拆分审核; 但它仍只是治理 packet, 不证明标签语义正确。R201 进一步做 policy sensitivity: 在 7 个 tail/split/generic-vocabulary 变体中, baseline review-required support 为 1.926%, 最大为 expanded generic vocabulary 下的 1.931%; lower/higher tail thresholds 使 long-tail support 在 0.921% 到 3.030% 间变化。这说明 review-required rows/support 在该 grid 中基本稳定, 但并未测量人工审核时间; higher-tail-threshold 会把 baseline head stability 降到 65.217%, 因此阈值仍是需要报告的展示策略风险。R202 用本机 llama.cpp server 对全部 41 个 regenerate/split candidates 跑候选重标, 得到 41/41 grammar-valid one-word outputs、0 invalid; 但这些输出仍只是候选, 不更新 canonical map。R202 的顶层 summary/attempts 面向公开 artifact, 嵌套的 r196-with-regeneration/ 是 local-audit-only, 因为其中可能包含 path/profile buckets。R203 进一步把这些候选变成 promotion review protocol: 41 行 promotion packet、两份空 reviewer sheets、0 final labels、long_tail_promotion_review_supported=false, 且 canonical_map_updated=false。R205 把这套机制转化为 reviewer 可检查的 compaction metrics: raw unique tag strings 从 1,546 降到 canonical unique tag strings 1,364, top-20 support coverage 从 93.683% 到 95.186%, long-tail support 为 1.746%, review-required support 为 1.926%; 同时 R203 final labels 仍为 0, R190 over/under-merge rates 仍为 n/a。R209 进一步导出可逆 display-map artifact: 1,811/1,811 raw tag rows 有 active display rows, active display labels 为 1,509, 63 个 deterministic alias rows 是 active display merges, 168 个 lexical/profile merges 仍是 pending merge candidates, 41 个 regenerated labels 仍为 candidate-only, reviewed display-map diff 为 0 行, 隐藏 other rows 为 0, drilldown support preserved, 且 drilldown CSV 包含每个 display bucket 的完整 raw-tag membership。R212 对这个 session/prompt 展示策略

做 ablation: raw、alias-only、profile-guarded-candidate-applied 和 R209 conservative display 四个变体都保持 183,714 total system-effect weight 不变; raw 有 26,829 个 system stacks, alias-only 和 R209 conservative display 都有 26,612 个, 假设激活所有 profile-guarded candidates 则为 26,067 个。但这个假设视图会在 2.532% 的 total effect weight 上激活未人工审核的 profile merges。因此 R212 支持的是 R209 的保守默认策略: alias 可以 active, profile merge 在人工标签回来前必须 pending; 它还没有覆盖 LLM/token display compaction。R213 进一步验证 display-mode data-layer contract: raw/display/pending 三种模式都保持 482,398 support, raw mode 有 1,811 个 buckets, display 和 pending mode 都有 1,748 个 buckets; pending mode 只叠加 209 个 candidate rows 和 323 个 review-required rows, 不改变 display membership, 且 drilldown membership 精确匹配 active display map。这组数字说明可逆 display overlay 能把长尾展示为可审计的聚合视图、提供 raw drilldown 并量化人工审核负担, 但它没有执行前端 renderer, 也不证明合并正确或标签足够好。R214 把这个长尾策略进一步变成控制环: 默认 view 保持 active-alias-only with pending overlays, 63 个 deterministic alias rows active, 168 个 profile-merge candidates 和 41 个 regenerated/split candidates pending, 323 个 rows 需要 review, 0 个 candidate merges active。它还输出一个非默认的 seven-bucket rollup preview, 将 1,811 个 raw-tag rows 和 482,398 support 按 governance state 精确分区; 同时记录 regeneration version policy, 当前 41 个 grammar-valid candidates 在没有 R203 human promotion labels 时有 0 个 promotable rows。更重要的是, R214 当前失败两个 gate: prompt-level review-required support 为 3.258%, 超过 3% 预算; higher-tail-threshold 下 head stability 只有 65.217%, 低于 80% gate。因此系统可以建议长尾合并或重生成, 但不能为了让火焰图更干净就自动提高阈值或推广 prompt-level candidates。R215 进一步检查前端 renderer-model 是否遵守同一契约: frontend/src/utils/agentflameDisplayModes.ts 在 TypeScript 下编译, 并由 Node harness 渲染 R209 display/drilldown rows, 同时 cross-check R213/R214 summary counts。该 smoke 保持 482,398 support 不变, raw/display/pending buckets 为 1,811/1,748/1,748, pending 只叠加 209 个 candidate rows 和 323 个 review-required rows, 0 个隐藏 other rows。两个负例也被拒绝: 错误 raw drilldown membership, 以及把 candidate display tag 当成 active membership。这说明 display contract 已被前端 TypeScript 模型消费, 但仍不是 browser DOM、visual click path、merge quality、adequacy 或 user utility 证据。R216 接着把同一 display contract 放入真实 headless browser: 脚本把该 TypeScript 模块编译成 browser ES modules, 启动临时 localhost DOM harness, 用 Chrome 点击 raw/display/pending 三个模式, 并保存 DOM dump 与 screenshot。浏览器可见状态仍保持 482,398 support, pending view 显示 1,748 buckets、209 candidate overlays, 323 review-required rows 和 63 active merges; 同样的 corrupted drilldown 与 candidate-promotion 负例也被拒绝。这支持的是 browser DOM harness smoke, 而不是 visual drilldown、merge quality、adequacy 或 user utility 证据。R217 进一步构建真实 Next static frontend, 提供一个由 R209 artifacts 支撑的最小 AgentFlame API fixture, 并在 headless Chrome 中打开 /agentflame。

生产 `AgentFlameView` 默认显示 `display mode`, 包含 1,748 buckets、482,398 support、3 个 mode buttons, 且 `display/drilldown membership` 匹配。这补上 production default rendering smoke, 但仍没有点击 production controls 或验证 visual drilldown。R218 则把长尾 merge/regenerate 的 promotion 机制变成 checked gate: 它用真实 R209 pending rows 构造 synthetic review fixtures, 接受 2 个 final consensus/adjudicated preview diff rows, 拒绝 4 个 unsafe rows (unclear, weak single-label, hidden other, missing source), 保持 1,811 raw keys 和 482,398 support 不变, 并保持 `canonical_map_updated=false`。因为 R218 的 review labels 是 synthetic, 它只证明 reviewed display-map diff gate 的机制, 不证明 promotion quality。R219 进一步把当前证据状态机械化成 claim/RQ readiness gate: C1 为 supported, C2 为 syntax/latency supported, C3 为 mechanism supported, C4 为 controlled live lineage plus generated-history audit (live lineage 由 R114/R191/R229/R232/R234 支撑; R230–R233 只是 generated full-history projection/normalization audit), C5 为 unsupported, C6 为 syntax/stability/protocol partial 且 human adequacy unsupported, C7 为 partial。该 gate 读取已生成 artifacts, 不读 raw traces, 不调用 LLM; 它记录 C5 responses 为 0、C6 final adequacy labels 为 0, 并保持 `weak_accept_supported=false`。因此 R219 的价值是把下一步固定为 P0 的 R142 participant returns 和 R124 human labels, 而不是提升论文 claim。R193/R194/R195 已把 R203 sheets 纳入 collection、preflight 和 scoring flow; 默认 run 仍没有返回标签, promotion、map update、C5 和 C6 都为 false。R242 进一步用 synthetic R124/R190/R203 sheets 验证这些 scorer 一旦收到完整输入就能被 R195 调用, 同时 partial/no-input/invalid-return cases 不会升级 canonical empty gates; 这仍不构成人工 adequacy 或 merge-quality evidence。R243 将这些空白 sheets 变成静态 labeler forms, 并保留 R195 文件名、source hashes、row counts 与 false gates, 因此它只是收集可用性证据, 不是标签质量证据。R244 再验证这些 forms 的静态加载与 CSV export shape; labeler smoke exports 仍保持 label cells 为空, 因此不改变 C6。R247 把这些 forms 打包为离线 distribution bundle, 并列出 `r124-labeler-1.csv`、`r124-labeler-2.csv`、`r190-labeler-*` 与 `r203-labeler-*` 的 R195 返回文件名; 这只是收集入口, 不是 adequacy 或 merge-quality evidence。OSDI 版本还需要 R124 人工 adequacy labels。R184 当前把 C6 记录为 `ready_for_independent_label_collection`, 不是 adequacy supported; R186 将 R124 labels 排在 R142 pilot 之后或并行执行。一词标签应该被定位为 lossy navigation frames, 而不是 semantic truth。

4.7 RQ6: 开源 artifact 路径是否可用?

R160 检查的是 artifact usability, 而不是 C5 用户效用或 C6 标签正确性。我们用 8 个固定历史 Codex session 文件运行 Rust CLI, 避免当前活跃 session 在 clean/cached 两次运行之间继续增长。Clean run 写入 `.agentsight/agentflame/r160-smoke-fixed`, 生成 dashboard、folded stacks、SVGs 和 `tags.json`, 在 76 个 tag requests 中发起 60 次真实 llama.cpp calls, 耗时 1.64 s。Cached rerun 使用同一组 `--session-file` 输入和同一个输出目录, 76/76 tag requests 命中 cache, 0 次 LLM call, 耗时 0.11 s。artifact_usability_r160.py

进一步检查 expected files、folded total equality、redacted previews、generated report path containment、dirty raw-trace-like paths、sanitized input manifest 和 clean/cached input equality; 结果写入 `docs/visexp/out/artifact-usability-r160.json`。

R200 进一步把输入换成临时 public-safe Codex fixture, 并由脚本管理 llama.cpp server。Clean run 发起 5 次真实 tag calls; cached rerun 为 0 次模型调用和 5/5 cache hits。该脚本显式记录没有读取真实 `.codex/.claude` traces, 临时 fixture 在运行后删除, committed summary 中 0 个非 redacted prompt previews, 且没有新增 raw-trace-like dirty paths。

R220 进一步把用户入口切到产品化的 `agentpprof`, 并从临时 clean clone 运行。脚本在 clone 内创建 public synthetic Codex fixture, 使用 deterministic regex tagger, 不调用 LLM, 也不读取真实 agent histories。它生成 `tasks.pb.gz`、`tools.folded`、`tokens.json`、`files.folded`、`network.folded` 和 `tools.svg`; 五个 projection 的样本数为 6/4/190/3/1; go tool pprof -top 能读取 `tasks.pb.gz` 并报告 6/6 total samples; tools/files/network/token 的 expected-stack checks、output containment 和 privacy scan 均通过。R248 将同一思路推进到 installable path: 先用 `cargo install --path agentpprof --locked --force` 安装本地包, 再用安装后的 `agentpprof` 读取 committed public fixture, 并生成同样的 pprof/folded/JSON/SVG outputs; go tool pprof -top 仍能读回 6/6 task samples, 且命令显式使用 `--session-file`、`--tagger regex` 和 `--no-cache`, 因此不读取私有历史、不调用 live tagger/model。R253 进一步验证 GitHub branch install path: `cargo install --git` 指向 `research/semantic-flamegraph-artifacts` branch, 并带上 `--locked --force agentpprof` 后成功; installed help 通过, 并在同一 public fixture 上生成 pprof/folded/JSON/SVG outputs; go tool pprof -top 读回 6/6 task samples, expected projection、output containment 和 privacy scan 均通过。这里的 no LLM calls 指 R253 driver 没有调用 live tagger/model; public fixture 本身仍包含 synthetic LLM-call frames。R254 去掉 branch 可变性: `cargo install --git --rev c43daf2b2565531dfd95de8654adabb30ac878d4` 安装固定 GitHub revision, install rev 与 driver commit 完全一致, repo dirty 为 false; 同一 public fixture 的 pprof/folded/JSON/SVG outputs、expected projection、privacy scan 和 go tool pprof -top 6/6 readback 仍全部通过。R256 则验证 crate package 边界: 在 clean provenance commit `a388c89ce718849ebfa5b8610709cbb50cf66b48` 上运行 `cargo package --list` 和 `cargo package`, 得到 8 个 intended package files, 生成 35,438 bytes 的 `agentpprof-0.2.0.crate`, archive/list 完全一致, 且 verify log 观测到 registry dependency `agent-session v0.3.3`; `target/`、`docs/visexp/out/`、私有 agent history 目录和主仓库其它组件都不在 crate 内。

这些结果说明 bounded local artifact path、public-safe fixture path、本机 clean-clone `agentpprof` pprof readback、本地 package install、GitHub branch install、pinned-revision GitHub install 和 crate package dry-run 在固定输入下是可审计、可重复的。但它们不等于 crates.io publish/readback、external-machine install, 也不等于社区开发者已经能顺利使用; 本地 `.agentsight` report 仍包含 trace

roots/session metadata, 因此不是 public-release artifact。一个更大的 36-session discovery run 首次标注耗时 173.05 s; 其 cached rerun 又出现 34 次新 LLM call, 因为当前 Codex session 在两次 discovery 之间增长。这暴露出默认体验问题: artifact smoke 和 cache benchmark 必须固定输入, 而交互式工具的默认模式应该明确采样、缓存和活跃 session 漂移。

4.8 评估完整性审计

表 4 把当前证据按 claim gate 汇总。这个表的作用不是装饰, 而是防止论文把 smoke evidence 写成系统结论。OSDI reviewer 最可能挑战的点是 C4、C5 和 C6: C4 已经有固定 command-mode suite、external cross-repo, 以及一次由 1 个 Claude command-mode task 加 2 个 Codex HTTP-family target-network probes 组成的受控扩展证据, R235 又证明单进程 Codex raw TCP 可以被 join; R238 进一步修复 process-tracer readiness race, 使 direct HTTP 和 direct multiprocess controls 在 compact direct-only readiness supplement 以及正式 full run 中通过; R240 又把 command-root over-attribution 与 target-child network capture 变成 regression tests。但更宽的 capture boundary 仍未关闭: R238 full run 的 Codex/Claude-launched rows 仍有 3 个 target-network orphan/missing-action cases。因此还要证明这些 agent-launched capture 边界、更多 agent family 和任意仓库 scope 不是过窄; C5 要证明用户能更快或更准确地回答 forensic questions; C6 要证明一词标签不只是语法稳定, 而是足够可用。R184/R186/R187/R188 的作用是防止过度声明: 它们把当前状态固定为 `not_weak_accept`, 把 launch material 与 outcome evidence 分开, 并把下一步执行顺序限定为 R142 pilot、R124 labels、R151 paper run。R245 在 R244 之后再次机械检查这个边界: 它读取 generated gate artifacts 和当前 paper/docs, 不读 raw traces、不调用 LLM; 9/9 个 hard evidence checks、13/13 个 wording checks 通过, 且 forbidden strong-claim hits 为 0。它同时记录 R219 已经是较早的 readiness board, R238/R240/R242–R244 必须作为 post-R219 addenda 或通过 R245 一起阅读。R246 进一步记录 post-R245 OSDI review 的 author response: C5/C6 仍是 must-fix outcome blockers, 同时把 R170 标成 dirty-provenance evidence, 并补充 R224 是 `checker_id=R131` 在 R170 denominator 上的 rerun, 而不是新的 checker 或 outcome evidence。因此当前最诚实的定位是 “supported mechanism plus partial systems artifact”。

4.9 R183: 为什么降级本身重要

R183 的价值在于防止把 “有网络事件” 写成 “目标 workload 的网络事件已被 lineage 捕获”。R182 的低层 codex network rows 已经被 join, 但 target-specific loopback/child-process rows 仍为 0/0。因此 gate 被降级为 target-specific oracle: 只有观察到并 join 目标 loopback 或 expected child-process network rows, 才允许把 C4 扩展到 network workload coverage。R191 正是该 gate 的后续实例: 它没有把 R182 的低层 agent runtime rows 当作充分证据, 而是要求固定 Codex task 执行预置 Python HTTP probe, 并以 4/4 target rows joined、0 negative joins 作为升级条件。R229 则把同一逻辑用于 controlled multi-workspace replication: 不同 workspace/category 必须分别通过 scoped lineage 和 negative-control gates; 其 raw join 为 36.481%, 这不是失败, 而是因为 out-of-scope 与 negative-control rows 应保持 orphan, 不能用 aggregate raw join rate 冒充完整 provenance。R230 把 gate 推到

full-history projection 层: 所有 folded system-observation weight 都必须带有 semantic session/prompt tag frames 和 call/effect frames, 且 raw tool events 的 prompt-index 必须可解析; 该 gate 通过, 但 duplicate prompt-index rows 和 0.188% tool-weight prompt-tag drift 仍存在。R231 随后证明 display projection 本身与 event-local prompt tags 完全一致, 并把 field-index drift 定位到 duplicate-index Claude sessions; R232 则把 live lineage replication 推到 AgentSight 仓库外的 controlled workloads。R233 最后把 prompt-row key 语义规范化: 裸 index 只在唯一时作 key, duplicate index 显式 non-keyed; 归一化后 semantic drift 为 0, 但 same-tag duplicate row identity 不再被伪装成可证明 lineage。R234 进一步把同一 oracle 用到 1 个 Claude command-mode task 和 2 个 Codex HTTP target-network probes 上, 说明 gate 可以承载 agent family 与 network probe 的局部扩展。R235 则把这个 gate 推到 raw TCP、multiprocess TCP 和 Claude-launched target-network, 结果只有单进程 Codex TCP 通过; R236 进一步把边界缩小为两类问题: Claude-launched delayed HTTP 仍没有 target rows, 而 direct controls 虽能观察 target rows, 但仍有 orphan。R237 加入 runtime witness 后, 4/4 witnesses 通过, 使 failure 不能再简单解释为 “probe 没有运行”。R238 修复 record -- 的 process-tracer readiness race 后, direct multiprocess control 稳定通过, 但 Codex/Claude-launched rows 仍有 target-network orphan/missing-action cases。R240 将这一路径中的 command-root fallback 和 target-child network capture 写成 regression tests, 说明系统现在能防止一类 over-attribution 回归。这说明 gate 有能力暴露 capture 边界, 而不是把 partial workload 包装成支持性证据; 下一步应针对 agent-launched process-time matching 和 Claude/Bun thread/process attribution 继续缩小边界。这体现本文的系统性: AgentFlame 不是把 trace 与 process summary 并排展示, 而是把 claim 也放入同一 lineage model 下审计, 区分用户任务 effect、agent runtime 噪声和必须保持 orphan 的事件。

5 相关工作

Flamegraphs 与 profiling. Brendan Gregg 的 flamegraph 让用户通过宽度定位 hot stacks。Grafana/Pyroscope、Datadog、Sentry 等系统已经把 flamegraph 用于 CPU、内存、profile 和 traces。AgentFlame 借用 folded stack 表示, 但 stack frame 来自 agent semantic context 与 system-effect lineage, 而不是函数调用栈。

Distributed tracing 与 agent observability. OpenTelemetry/Dapper 风格 tracing 把一次请求表示为 span tree。LangSmith、Langfuse、Phoenix、AgentOps 等工具已经记录 agent、LLM、tool、retrieval、cost 和 latency。SigNoz/Inkeep 公开展示了 multi-agent workflow 的 span-duration flamegraph。这些系统擅长解释单次 workflow 如何执行; AgentFlame 的目标是跨 session 聚合 “哪个语义任务造成哪些系统副作用”。

Token/context profiling. Context 和 token profiling 工具关注 prompt/context window 中的热点。AgentFlame 包含 token flamegraph, 但当前只作为 source-local accounting。论文主贡献是 token/semantic/tool/process/effect 共享同一 folded-stack 语法, 而不是 token cost 本身。

表 4: 当前 claim gate 的主文摘要。完整证据 ledger 保存在 CLAIM_VERDICT.md; 这里仅保留 reviewer 需要的 status、最强证据锚点和下一步 gate。

Claim	Verdict	Evidence anchor	Next gate
C1 folded aggregation	supported	R170 folds 183,714 system observations into 26,829 semantic stacks with folded totals matching the report.	Fresh-clone release workflow and public packaging.
C2 one-word tags	syntax supported; adequacy open	R170 tags 2,859 prompt rows and 114,837 LLM-call events with 0 final tag failures; R180 shows local 0.6B/1B/3B-class syntax/latency stability.	Human adequacy labels, not grammar validity.
C3 semantic partitioning	mechanism supported	R170/R224 show nonsemantic/flat mixed weight 90.402%/90.918%; prompt-only reduces system bucket/residual mixed weight to 36.722%/7.485%; R209/R212 keep display compaction reversible.	More repositories, user tasks, token/display ablations, and human-reviewed merge/promotion quality.
C4 exact lineage	fixed/controlled workloads supported; broad scope partial	R114/R191/R229/R232/R234 give scoped precision/recall evidence with zero negative joins; R238/R240 add direct-process readiness and regression guards while exposing Codex/Claude-launched target-network orphan cases.	Agent-launched network boundary, same-tag prompt-row identity, more agent families, and richer target network workloads.
C5 user utility	unsupported	Outcome evidence: none yet. Protocol/logistics readiness includes R142/R187/R207/R249/R255/R258 and the R259 paper-scale static collection kit; there are still 0 participant responses.	Score real paper-scale participant responses under the frozen preregistration.
C6 tag adequacy	syntax/protocol partial; adequacy unsupported	Outcome evidence: no human adequacy labels yet. R180/R251 provide syntax/stability and behavior-association evidence; R252 plus R258/R259 static forms only prepare the C6 label handoff.	R124/R190/R203 independent human labels and adjudication.
C7 artifact usability	partial	R160/R200/R220/R248/R253/R254/R256 cover bounded local artifacts, public fixture, local install, GitHub install, pinned revision, pprof readback, and crate-package dry run.	crates.io publish/readback, external-machine install, sanitized real report, setup docs, write-set audit, and external developer feedback.

6 局限与下一步

当前版本仍不到 OSDI weak accept。Exact lineage 是最强系统贡献，但证据分两类：live lineage 由 R114 的 20 个固定 command-mode Codex 任务、R191 的固定 Codex target-process HTTP workload、R229 的 5 个 controlled multi-workspace tasks、R232 的外部 fresh git-repo/HTTP-probe workload, 以及 R234 的 1 个 Claude command-mode task 加 2 个 Codex HTTP-family target-network probes 的受控扩展支撑；R182 只是 record-mode --trace-net 的低层 agent-process network smoke。R235–R240 新增 raw/Claude target-network 边界实验与 regression guard：R235 只让单进程 Codex TCP 通过；R236 显示 Codex delayed multiprocess 可以通过，但 Claude-launched delayed HTTP 仍没有 target rows，direct controls 仍有 orphan rows；R237 进一步加入 runtime witness；R238 修复 process-tracer readiness race 后，compact direct-only supplement 汇总 5/5 repetitions 通过，正式 full run 中 4/4 witnesses 与 4/4 witness ports observed、direct controls joined、13/16 target network rows joined、0/186 negative-control effects joined，但 Codex/Claude-launched rows 仍 partial；R240 将 command-root fallback 和 target-child network capture 加入 regression suite。因此它们缩小并定位 C4 缺口，不把广义 raw-socket/Claude-launched coverage 升级为 supported；direct-only repetitions 没有 negative controls，不能替代正式 full run 的 precision evidence。R230–R233 支撑的是 generated full-history projection correctness：R230 证明 system-effect stacks 都带有 semantic session/prompt tag frames 和 call/effect frames，R231 证明 display projection 与 event-local prompt tags 一致并定位 prompt-row drift，R233 把 duplicate prompt indexes 显式降级为 non-keyed row identity 并把 normalized semantic drift 降为 0；但 same-tag prompt-row identity、R238 暴露的 Codex/Claude-launched target-network orphan/missing-action 边界、任意仓库泛化、更多 agent 类型和更丰富的 network workloads 仍缺。用户效用也没有 outcome data：R142 已冻结五种视图的任务

包、answer key 和 scorer，R187 已生成可发放的 P01–P05 pilot launch package，R207 已确认 launch handoff 和 R195 return-file mapping，R249 已生成 12 个 participant-packet paper-scale launch package，R255 验证 R249 assignment 可以通过 R195 且旧 R142 assignment 会失败，R258/R259 又把 paper-scale bundle 和 static collection forms 做成可发放路径，但仍需要真实参与者响应；纸面 C5 还需要 12–20 个参与者或明确限定的 expert study。标签 adequacy 仍需真实人工返回：R252 已把 R124 adequacy、R190 merge-risk、R203 promotion review 打包为 2 个 labeler packets、每个 501 行，并验证 blank R195 input 仍保持 gates false；R258/R259 只是把这些 labeler packets 放进 unified bundle 与 static collection forms，仍需要真实 labeler sheets。R251 只说明 prompt tags 与 sanitized system behavior 有 weighted association，并且 p-value 不是 independent-session inference；R189/R190/R190-score/R196/R201/R202/R203/R205/R209/R211/R212/R213/R214/只能说明 canonical display layer、merge-risk audit protocol、long-tail governance、policy sensitivity、候选重标链路、promotion gate、compaction metrics、可逆 display-map contract、RQ2 figure/example inputs、display-compaction ablation、display data-layer smoke、long-tail control loop 与非默认 rollup/regeneration policy、frontend renderer-model smoke、browser DOM harness、production default render smoke 和 synthetic-review update gate 已具备，不能替代人工 adequacy、promotion quality、visual drilldown 或真实 user evidence。R160 验证本机固定输入下的 bounded local artifact path，R200 验证 public-safe fixture path，R220 验证本机 clean-clone agentpprof pprof readback path，R248/R253/R254 分别验证本地 package install、GitHub branch install 和 pinned GitHub revision install path，R256 验证 agentpprof 0.2.0 的本地 crate-package dry-run；但 crates.io publish/readback、external-machine 安装、真实 report public sanitization、llama.cpp LLM-tag setup、默认采样策略、full write-set audit 和外部开发者反馈仍缺。R245 说明当前文本没有把这些缺口写成 supported claims；R246 进一步修复

R170/R224 provenance bookkeeping, 但它们本身仍只是 audit hygiene。R250 在 R249 后再次记录两个独立 review: R249 被接受为 paper-scale launch readiness, 但文档必须写成 participant packets 而不是真实 participants, R249 completed responses 应进入私有 CSV, R248 生成报告需要 redact 本机路径并扫描 manifest; 这些修订仍不改变 weak-accept gate。

7 结论

AgentFlame 的核心不是再画一张 agent flamegraph, 而是把用户语义意图、LLM/tool 历史和系统副作用放入同一个可折叠、可聚合、可审计的栈模型。当前 full run 说明该模型在真实本地 agent 历史上可运行, 并且语义帧能分开 nonsemantic 和 flat baselines 大量混合的系统效应。R114 进一步说明固定 command-mode suite 中 exact semantic-effect lineage 可以做到高精度, 并拒绝并发 negative controls; R182 说明同一路径在启用 record-mode `--trace-net` 后能 join 低层 agent-process network rows; R191 说明固定 Codex target-process HTTP network rows 也能被 join; R229 说明同一 oracle 能在 controlled multi-workspace tasks 上复现; R232 说明该 oracle 能迁移到 AgentSight 仓库外的 controlled git-repo 与 HTTP-probe workload; R234 说明它还能承载 1 个 Claude command-mode task 和 2 个 Codex HTTP-family target-network probes 的受控扩展; R235–R238 说明 raw/Claude target-network scope 仍是 partial, R238 在受控条件下修复 direct multiprocess readiness, 但 Codex/Claude-launched target-network orphan/missing-action 仍要继续修; R240 把 command-root over-attribution 和 target-child network capture 写入 regression guard; R230 说明 full-history projection coverage 可审计, R231 说明 display projection 与 event-local prompt tags 一致, R233 说明 prompt-row semantic consistency 可通过 non-keyed duplicate-index normalization 成立。R160、R200 和 R220 说明 bounded artifact path、public-safe fixture path、cache rerun 和本机 clean-clone pprof readback 已经可审计, 但还不是社区级 artifact 结论。但顶会版本必须继续证明 R238 暴露的 Codex/Claude-launched target-network 边界、更多 agent family 和任意仓库 scope、same-tag prompt-row identity policy、标签 adequacy 和用户任务收益。在当前证据下, 最诚实的论文定位是: 一个有完整系统化方向和真实 characterization 的语义系统效应 profiler, 而不是已经完成的 OSDI artifact。

参考文献

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] Benjamin H. Sigelman et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report, 2010.
- [3] SigNoz. Understanding Flame Graphs for Visualizing Distributed Tracing. <https://signoz.io/blog/flamegraphs/>.
- [4] SigNoz. How Inkeep Monitors Their AI Agent Framework with SigNoz. <https://signoz.io/blog/inkeep-ai-agent-monitoring/>.
- [5] Datadog. Trace View. https://docs.datadoghq.com/tracing/trace_explorer/trace_view/.
- [6] LangChain. LangSmith tracing documentation. <https://docs.langchain.com/langsmith/>.
- [7] Langfuse. Observability documentation. <https://langfuse.com/docs/observability>.
- [8] Arize Phoenix. Tracing documentation. <https://arize.com/docs/phoenix>.